Running AV Reasoning Models Within Onboard Compute and Latency Limits: 분석 리포트

게시용 버전 · 원문 https://developer.nvidia.com/blog/running-av-reasoning-models-within-onboard-compute-and-latency-limits/ · NVIDIA 기술 블로그 (Marco Pavone, Boris Ivanovic) · 게재 2026-11-10 · 접근 2026-11-12 · **동료심사 없음** · 블로그 그림은 싣지 않고 자체 그래프만 씀 · 저장소 라이선스: 코드 Apache-2.0, 가중치 OpenMDW-1.1

핵심 요약

1. **무엇인가:** Alpamayo 2 Edge(공식 config로 세면 2.65B: 백본 2.45 + expert 0.2)를 DRIVE AGX Thor에 올려 블로그가 160-190 ms라고 보고했습니다. AlpaJudge는 추론 판정기(RB·RF)로, 데이터 선별과 RL 보상에 씁니다.
2. **파라미터·점수식은 맞다:** config 치수로 센 값이 모델 카드와 맞고, AlpaJudge RF 식과 노트북 기록값 0.842도 재계산과 일치합니다.
3. **지연 수치에는 조건이 빠져 있다:** 레시피 단계 시간을 더하면 165 + 12r ms입니다. 블로그의 160 ms 하한은 라운드 0개일 때보다도 낮습니다. 롤링 대 콜드 스타트(약 +53 ms)와 Euler 5 대 10단계(+70 ms)가 같은 크기입니다.
4. **"전 과제 우위"는 아니다:** 페루프 중대 사고 간 거리는 약 60배지만, LingoQA는 Edge 75.2 대 Qwen 79.4로 낮습니다.
5. **디코드는 대역폭에 묶여 있다:** 레시피의 7 ms/토큰은 약 231 GB/s의 실효 대역폭이고, 273 GB/s 가정의 약 85%입니다. 바이트를 줄이는 것 말고 빠른 길이 없습니다.
6. **추측 디코딩은 수락률 0.42 이상일 때만 이득이다:** 초안 모델(DFlash2)은 릴리스에 없어서 블로그 수치를 재현할 수 없습니다.
7. **구현 가능성:** 이 맥(M4 Pro 24 GiB, CUDA 없음)에서는 Thor 시스템을 구현할 수 없습니다. 모델 PyTorch 추론은 메모리상 들어가고 CUDA 호출을 고치면 **조건부로 포팅**할 수 있지만, 수 초가 걸릴 것으로 보여 지연 목표와는 거리가 멉니다.

01. 블로그 분석: Running AV Reasoning Models Within Onboard Compute and Latency Limits

> 이 글은 논문이 아니라 **NVIDIA 기술 블로그**입니다. 동료심사를 받지 않았고, 제품 홍보 성격이 섞여 있습니다. 아래 수치는 모두 블로그 또는 같은 주장에 해당하는 공개 자료에서 가져왔고, 출처를 붙였습니다.

0. 자료 정보

항목	내용
제목	Running AV Reasoning Models Within Onboard Compute and Latency Limits
필자	Marco Pavone, Boris Ivanovic (NVIDIA)
게재일	2026-11-10 (블로그 `article:published_time` 2026-11-10T16:00Z)
접근일	2026-11-12 (추적 파라미터 없이 `https://developer.nvidia.com/blog/running-av-reasoning-models-within-onboard-compute-and-latency-limits/`)
저장	`paper/blog_raw.html` (원본 HTML), `paper/blog.txt` (본문 추출)
성격	NVIDIA 기술 블로그. 동료심사 없음. 본문 상단에 AI 생성 요약이 붙어 있음
권리	블로그에 명시된 라이선스 문구가 없습니다. **저작권이 NVIDIA에 있다고 보고** 그림은 게시하지 않았고, 본문은 짧게 요약·인용했습니다. 게시본에는 블로그 그림이 없고, 모든 그림은 자체 계산으로 그린 것입니다

블로그가 가리키는 대상은 **Alpamayo 2 Edge**(2B 추론 VLA, DRIVE AGX Thor 대상)와 **AlpaJudge**(추론 품질 판정기)입니다. 블로그에서 가장 구체적인 주장은 "Thor에서 픽셀에서 궤적까지 160-190 ms"라는 지연 수치입니다.

1. Abstract에 해당하는 주장

블로그 상단 요약과 서론이 말하는 내용은 다음과 같습니다.

1. Alpamayo 2 Edge는 **2B 파라미터**의 적응형 추론 VLA입니다. 백본은 Cosmos 3 Edge의 Nemotron 계열 2B 추론 백본이고, 궤적은 확산 행동 전문가(diffusion action expert)가 만듭니다.
2. 추론(Chain-of-Causation, CoC)은 **필요할 때만** 생성합니다(적응형 추론, 블로그 표현으로 "NOMINAL" 토큰).
3. **폐루프 강화학습**을 AlpaGym에서 해서 폐루프 주행 성능이 "최고 체크포인트 대비 48.9%" 좋아졌다고 주장합니다.
4. AlpaJudge는 사람이 보정했다고 하는 추론 평가·보상 모델입니다. RB(참조 기반)와 RF(참조 없음) 두 체크포인트가 있습니다.
5. 배포 레시피(TensorRT, TensorRT-Edge-LLM, DFlash2 추측 디코딩)로 **Thor에서 160-190 ms**를 얻는다고 합니다.

2. Method: 블로그가 설명하는 구성

2.1 모델 구조와 크기

- 백본: Cosmos 3 Edge 기반 Nemotron 계열 2B 추론 백본. 공개 config로 세면 텍스트 1.963B(임베딩·LM head 포함), 비전 0.411B(SigLIP2형), projector 약 0.08B, 액션 expert 0.198B. 합계 **2.65B**이고 모델 카드의 "백본 2.45B + expert 0.2B"와 맞습니다(`03` §1, `config_check.py`).
- 입력: 6대 카메라, 22장 프레임. 전방 광각·망원(카메라 1, 6)은 5장(−2, −1, −0.5, −0.1, 0초), 나머지 4대는 3장(−1, −0.1, 0초). 카메라마다 픽셀 예산이 다름(전방 163,840-196,608, 나머지 81,920-98,304). 20개 과거 자세(−1.9-0초, 0.1초 간격)와 내비게이션 문장.
- 출력: CoC 텍스트(기본 최대 256 토큰)와 64 웨이포인트(0.1-6.4초, 0.1초 간격). 행동은 unicycle 가속도·곡률(accel, curvature) 공간.

2.2 학습 방법

- **오프라인 증류**: Alpamayo 2 Super가 만든 CoC 자동 라벨로 학습(블로그 주장, 데이터·코드 공개 여부 불명).
- **폐루프 RL**: AlpaGym, 수치 "+48.9%"(블로그 주장).
- **다해상도 입력**: 카메라별 픽셀 예산(위 2.1, 공개 config로 확인됨).

2.3 추론 최적화

- **추측 디코딩 DFlash2**: 블록 16 초안 모델로 토큰을 미리 제안하고 큰 모델이 한 번에 검증. 블로그는 "품질을 떨어뜨리지 않고 가속"이라고 함.
- **양자화**: TensorRT Edge-LLM의 FP8 보정(calibration). 레시피는 FP8 가중치, 활성값, KV 캐시를 쓰고 비전 앞 층은 FP16으로 둠.
- **엔진 구성**: 행동 expert는 TensorRT 그래프로 내보내고, Euler 적분 5단계를 씀. 공개 PyTorch 예제는 10단계(차이는 `04` D8).

3. Experiments: 블로그 수치

3.1 다른 모델과 비교 (블로그 Figure 2, 막대 값 판독)

Figure 2의 Edge 값은 모델 카드 수치와 맞습니다(Ego3D, VLADBench, SURDS, WaymoQA, nuReasoning, LingoQA, KITScenes). Qwen-Drive-1.0(4B) 값은 확인할 공개 자료가 없습니다.

항목	Alpamayo 2 Edge (2B)	Qwen-Drive-1.0 (4B, 블로그 값)	비고
폐루프: 중대 사고 간 거리 (km)	15.15	0.25	약 60배 (15.15/0.25 = 60.6)
Ego3D 정확도	84.9	61.0	모델 카드 84.94
VLADBench	69.9	66.5	모델 카드 69.94
SURDS	86.5	66.1	모델 카드 86.50
WaymoQA (all)	75.8	74.5	모델 카드 75.77
nuReasoning (choice)	78.6	58.4	모델 카드 78.58
LingoQA (LingoJudge)	**75.2**	**79.4**	Edge가 낮음
KITScenes (motion)	70.0	52.5	모델 카드 70.00

블로그 제목 "Outperforms Across Major AV Tasks"와 달리 **LingoQA에서는 Qwen-Drive가 더 높습니다.**

3.2 폐루프 주행 (모델 카드)

- 875개 시나리오, 시나리오당 3회 롤아웃. 중대 사고 사이 주행 거리 **15.15 ± 0.04 km**(평균 ± 표준편차, 3세트). AlpaSim 기반.
- 개방 루프: 1,392 샘플, minADE_6@5초 **0.52 m**(모델 카드).
- 블로그의 "+48.9% (RL)"에 해당하는 표·수치는 공개 자료에서 찾지 못했습니다.

3.3 RL 보상의 효과 (블로그 Figure 5, 판독)

보상	자체 판정기 기준 향상	사람 평가 기준 향상
AlpaJudge 보상	+10.2%	+11.4%
기준선 보상	+13.4%	+1.8%

이 그림은 "자체 판정기로 잰 향상"과 "사람이 잰 향상"을 나란히 보여 줍니다. 기준선 보상에서 자체 평가는 13.4%인데 사람 평가는 1.8%라 차이가 큽니다. 이것은 판정기 자체가 보상에 맞춰지는 문제(reward hacking)가 있을 수 있음을 보여 주는 증거이기도 합니다. 다만 블로그는 이 해석을 쓰지 않았고, 사람 평가 절차와 표본 수도 공개하지 않았습니다.

3.4 데이터 선별 (블로그 Figure 4, 판독)

판정 점수 분포를 세 구간으로 나눕니다. 낮음(0-0.26)은 라벨 오류나 어려운 사례, 중간(0.26-0.66)은 학습 가치가 큰 사례, 높음(0.66-1.0)은 이미 해결된 사례라고 봅니다. 원자료에서 각각 16%, 27%, 57%이고, 중간 구간을 표본 추출하면 81%가 됩니다. 이 임계값은 그림에 적힌 값이지만 선택 근거는 쓰이지 않았습니다.

3.5 지연 (블로그 본문)

- "픽셀에서 궤적까지 **160-190 ms**"(추론량에 따라 다름). 측정 장치: DRIVE AGX Thor-X, DRIVE OS 7.2.7.0, CUDA 13.4, TensorRT 11.3.1.9.
- 측정 방법(몇 번 반복, 예열 여부, 롤링 또는 콜드 스타트, 샘플 수)은 블로그에 없습니다. 레시피 README에서 측정 조건을 찾았습니다(`04` §2).
- 파트너 인용: Embedl은 "AGX Thor에서 10 Hz 계획", PORT는 Alpamayo 1.5 서버보다 "약 2배 빠른 계획", Niulinx는 minADE 13.6% 감소·횡방향 오차 46.1% 감소. 모두 파트너의 자체 보고이고 검증하지 못했습니다.

4. Limitations: 블로그가 말하지 않은 것

블로그에는 한계 절이 없습니다. 다음은 공개 자료로 확인한 빈칸입니다.

1. **측정 조건**: 지연의 반복 횟수, 샘플 수, 롤링 또는 콜드 스타트, 소프트웨어 버전(레시피 README는 "지표용이 아닌 대략적 타이머"라고 씀).
2. **양자화 품질**: 블로그는 "품질 보존"을 말하지만 BF16 PyTorch 기준과의 비교 수치가 없습니다. 레시피는 FP16 엔진과 비교한 minADE_6만 줍니다(1.547 m 대 1.570 m, 내부 10 클립).
3. **RL 효과**: "+48.9%"의 기준선, 시나리오 수, 반복 횟수가 없습니다.
4. **AlpaJudge 보정**: "사람이 보정했다"고 하지만 보정 데이터, 일치도, 신뢰구간이 공개 저장소에 없습니다.
5. **NOMINAL 토큰**: 학습 데이터 준비 스크립트에는 `<NOMINAL>` 라벨이 있지만, 추론 때 이 토큰이 얼마나 자주 나오는지는 공개 자료로 확인하지 못했습니다(`04` C14).
6. **추측 디코딩 초안 모델(DFlash2)**: 블록 16 초안 모델은 "모델 릴리스의 일부가 아니다"라고 레시피가 명시합니다.

5. 이 저장소의 관심사와 연결

블로그는 AV 추론 모델을 온보드 예산에 맞추는 **배포 공학** 글입니다. 로봇 VLA·월드 모델 관점에서 얻을 것은 다음과 같습니다.

- 지연을 **단계별로 쪼개고 측정 조건을 함께 적는** 방식(롤링 대 콜드 스타트의 차이가 53 ms 정도로 추정됨, `03` §3).
- 추측 디코딩의 손익분기 수락률(약 0.42, `03` §5). 행동 토큰에서 이 값을 넘는지는 미지수입니다.
- 판정기 점수로 **데이터를 고르고** RL 보상을 만드는 흐름. 다만 블로그의 보상 향상 수치는 자체 판정과 사람 평가가 크게 엇갈립니다(§3.3).
- 적응형 추론(필요할 때만 추론): 평균 지연을 줄이는 효과를 모델로 계산할 수 있습니다(`03` §5). 블로그는 NOMINAL 비율을 보고하지 않았습니다.

02. 선행 연구, 공개 저장소, 관련 분석

1. 계보와 이 블로그의 위치

계열	무엇	이 블로그와의 관계	공개 여부 (이번 확인)
Alpamayo-R1 / Alpamayo 1.5	추론 VLA (10B), SFT·RL·추측 디코딩 레시피	전 세대. AlpaGym의 공개 정책이 이 모델(1.5, 10B, GPU 2장)	코드와 레시피 공개(alpagym, alpamayo-recipes)
Alpamayo 2 Super	교사 모델. CoC 자동 라벨 생성, SFT 레시피 있음	블로그가 증류 교사로 씀	레시피 공개(`alpamayo2_sft`). 가중치 여부는 미확인
Cosmos 3 Edge	2B급 Nemotron 계열 백본	Alpamayo 2 Edge의 백본 (config의 `cosmos3_edge`)	모델 카드의 base_model로 표시됨
Alpamayo 2 Edge	2B 추론 VLA, 확산 expert	**본 블로그의 대상**	가중치 (OpenMDW-1.1), PyTorch 추론 (Apache-2.0), Thor 레시피 (Apache-2.0)
AlpaJudge (RB, RF)	추론 판정기, 참조 기반·참조 없음	블로그의 평가·보상 모델	가중치 (OpenMDW-1.1), 추론 패키지 (Apache-2.0)
AlpaGym	페루프 RL, AlpaSim 기반	블로그의 "+48.9%" 학습 환경	공개 정책은 Alpamayo 1.5 10B만 있음. Edge용 RL 경로는 없음
Qwen-Drive-1.0 (4B)	블로그 비교 대상	비교 상대	공개 자료 확인 못 함
LingoQA / Lingo-Judge	주행 추론 평가 벤치마크·판정기 (Wayve)	모델 카드가 쓰는 추론 점수	공개 벤치마크 (이번에 확인하지 않음)
DFlash / DFlash2	블록 확산 기반 추측 디코딩	블로그의 초안 모델 방식	z-lab/dflash (MIT)
TensorRT-Edge-LLM	NVIDIA 엔진 빌드·런타임	레시피가 v0.11.1로 고정	Apache-2.0

블로그는 "Alpamayo 2 Edge는 2B"라고 쓰지만, 공개 config로 세면 전체는 **2.65B**입니다(백본 2.45B, expert 0.2B). 모델 카드도 이 구분을 씁니다.

2. GitHub와 Hugging Face 목록

확인일 2026-11-12. `gh api` 로 읽은 메타데이터와 커밋이고, 가중치는 받지 않았습니다. 설정 파일 두 개(config.json)와 README만 받았습니다.

저장소	역할	라이선스	확인 커밋 (날짜)	마지막 push	이번에 본 범위
NVlabs/alpamayo2-edge	2B 추론 PyTorch 코드, 샘플링 설정, 가속 경로	Apache-2.0 (가중치 OpenMDW-1.1)	`02f55a7` (2026-11-10)	2026-11-10	README, `config.py` , `inference.py` , `docs/acceleration.md` , `accel/decode.py` , `pyproject.toml` , 테스트 목록
NVlabs/alpamayo-recipes	Edge의 Thor 배포 레시피(FP8, TensorRT), SFT, 추측 디코딩, RL 그레이더	Apache-2.0	`622f9d2` (2026-11-11)	2026-11-11	`alpamayo2edge_edgellm` README· `thor/runtime/speculative.py` · `host/quantize.py` · `coc_vocab.json` , `alpamayo1_x_rl` 의 `BaseReasoningGrader`
NVlabs/alpajudge	AlpaJudge RB·RF 추론 패키지	Apache-2.0 (가중치 OpenMDW-1.1)	`607ae9d` (2026-10-29)	2026-10-29	`reference_free/model.py` (점수 식), 예제 노트북(기록된 출력), README 한계 절
NVlabs/alpagym	페루프 RL 프레임워크	Apache-2.0	`972d160` (2026-07-21)	2026-09-03	파일 목록. 공개 정책은 `alpamayo_r1` (1.5, 10B)뿐
NVIDIA/TensorRT-Edge-LLM	엔진 빌드·런타임 (레시피가 `v0.11.1` 고정)	Apache-2.0	미확인	2026-11-06	메타데이터만

저장소	역할	라이선스	확인 커밋 (날짜)	마지막 push	이번에 본 범위
z-lab/dflash	DFlash 블록 확산 추측 디코딩	MIT	미확인	2026-08-18	메타데이터만
HF nvidia/Alpamayo2-Edge	모델 카드, config.json	OpenMDW-1.1	리비전 `f8951d09` (PyTorch README), `aaaa32c` (Thor 레시피)	모델 카드 릴리스 2026-11-10	README, config.json (가중치 미다운로드)
HF nvidia/alpajudge	RB·RF 체크포인트	OpenMDW-1.1	미확인	미확인	README, config.json (가중치 미다운로드)

라이선스 메모

- 코드는 Apache-2.0이고, 모델 가중치는 OpenMDW-1.1입니다(모델 카드 기준). 두 저장소 모두 이 구분을 명시합니다.
- AlpaJudge 백본은 Qwen3-VL형 config로 보입니다(`model_type: qwen3_vl`, `backbone_kind: alpamayo1`). 상위 백본 라이선스는 확인하지 않았습니다.
- 블로그 본문과 그림은 NVIDIA 저작물로 보고, 그림은 게시하지 않았습니다.

3. 같은 저장소의 관련 분석 (블로그 §3 연결)

이 블로그는 AV 전용이지만 **온디바이스 추론의 배포 원리**는 기존 분석과 같습니다. 아래 분석의 산술과 판단 기준을 재사용했습니다.

기존 분석	가져온 것	이 블로그에 적용한 곳
`nemotron35_bolt/`	토큰당 읽는 바이트 = 가중치 + KV, 대역폭 상한(tok/s), MoE 초안 검증 비용, 24 GiB 맥 메모리 판정	`03` §4의 디코드 상한, `03` §2의 메모리 산술
`nemotron35_super_vl/`	NVIDIA 공식 서빙이 CUDA·NVFP4 커널 전용이라 맥에서 못 돌린다는 판정 기준	`04` §4의 맥 판정
`no_long_context/`	비용 모델(처리량·메모리 한계)을 구조별로 계산하는 방식	지연 예산과 추론 토큰 수의 관계(`03` §5)
`imle_vla/`, `adavla/`	행동 헤드 지연을 "스텝 수 × 한 스텝 비용"으로 보는 관점, 맥 MPS에서 행동 헤드를 돌릴 때 CUDA 그래프 의존성	Euler 5단계 대 10단계의 지연 차이(`03` §3)
`pacevla/`	행동 expert를 시간 해상도별로 나누는 설계, GR00T 기반 VLA의 재현 장벽	로봇 VLA로 옮길 때의 연결

4. 구성 요소별 출처와 검증 가능성

구성 요소	출처	검증 방법	검증 결과
백본·expert 크기	HF config.json, 모델 카드	config 치수로 파라미터 계산	일치 (2.649B)
입력 구성(22프레임, 픽셀 예산, 20 자세)	README, config.py, 모델 카드	정적 대조	일치
샘플링 설정(seed 43, top_p 0.98, temp 0.6, 256, 10단계)	블로그 코드, `inference.py` 기본값	정적 대조	일치
지연 160-190 ms	블로그	레시피 단계 시간과 대조	불일치 가능성 (`03` §3)
FP8 양자화·추측 디코딩	레시피 README, `quantize.py`, `speculative.py`	정적 대조	구현 존재. 초안 모델은 공개 안 됨
AlpaJudge RF 식	`reference_free/model.py`	식 대조, 노트북 기록값 재계산	일치 (`03` §6)
AlpaJudge RB 0.999	노트북 마크다운 기록	재계산 불가 (체크포인트 실행 필요)	확인 불가
페루프 RL +48.9%	블로그	공개 코드·수치 검색	확인 불가
NOMINAL 토큰	블로그	데이터 준비 스크립트 검색, 추론 코드·어휘 검색	학습 라벨은 존재. 추론 빈도는 확인 불가

03. 수치 재계산과 산술 검산

모든 계산은 CPU에서 수 초 안에 끝납니다. 결과 JSON은 `analysis/*_results.json`에 있습니다.

- `analysis/config_check.py` → `config_check_results.json` : 파라미터, 가중치·KV 바이트, 측정 메모리 산술
- `analysis/latency_toy.py` → `latency_toy_results.json` : 디코드 대역폭 상한, 지연 모델, 추측 디코딩 수학, NOMINAL 민감도
- `analysis/rf_score_check.py` → `rf_score_results.json` : AlpaJudge RF 점수 재계산

가정은 각 절에 적었습니다. 블로그나 레시피에 없는 값에는 "가정"이라고 표시했습니다.

1. 파라미터 산술

구성 요소	config로 센 값 (B)	모델 카드 (B)	판정
텍스트 디코더 (28층, d=2048, MLP relu2, 임베딩·LM head 분리)	1.963	1.96	일치
비전 인코더 (27층, d=1152, 2행렬 MLP)	0.411	0.41	일치
projector (2층 MLP 가정)	0.077	0.08	일치 (구조는 가정)
백본 합계	2.451	2.45	일치
액션 expert (28층, d=512, 16/8 헤드)	0.198	0.2	일치
전체	**2.649**	"2B-parameter"	명명 차이 (블로그의 "2B"는 공칭 크기)

AlpaJudge RF는 같은 방식으로 세면 약 **2.06B**(텍스트 1.73B, 비전 24층 0.30B, merger 0.03B)입니다. BF16으로 약 4.1 GB이고, 노트북과 README의 "가중치 약 5 GB"와 같은 자릿수입니다. 차이는 공개되지 않은 구조 세부 때문일 수 있습니다.

2. 메모리 산술

항목	값	계산
BF16 가중치	4.93 GiB	2.649 B × 2 B
FP8 가중치 (전부 FP8)	약 2.65 GB	2.649 B × 1 B
KV 캐시, 토큰당 BF16	114,688 B (112 KiB)	28층 × 8 KV헤드 × 128 × (K,V) × 2 B
프롬프트 3,236 토큰의 KV, 1벌	354 MiB	3,236 × 114,688 B
같은 KV, 후보 6벌	2.07 GiB	6 × 354 MiB
H100 측정 peak allocated	8.40 GiB (모델 카드)	
잔여분 (활성값·ViT·KV·샘플 버퍼)	3.47 GiB	8.40 − 4.93

- 잔여분 3.47 GiB 안에 후보 6벌의 KV(2.07 GiB)가 들어갑니다. 활성값과 ViT 버퍼를 생각하면 빠듯하지만 모순은 없습니다. 후보마다 KV를 따로 두는지는 구조가 공개되지 않아 추정입니다.
- 이 맥(통합 메모리 24 GiB)에서 BF16 가중치는 약 21%를 차지합니다. **메모리 용량은 문제가 되지 않습니다.** 문제는 대역폭과 연산 시간입니다(§3-§4).
- 모델 카드의 "최소 16 GB GPU 메모리" 권장과도 맞습니다.

3. 지연 모델 (레시피 단계 시간 합)

레시피 README의 Thor 측정표(예시 클립, 22프레임, 약 3,300 프롬프트 토큰, 드래프터 4 라운드, 6 후보, Euler 5단계)입니다.

단계	시간 (ms)	비고
ViT (최신 6프레임만)	20	롤링 측정. 나머지 16프레임은 시계 시작 전에 인코딩
프리필	75	
추측 라운드 1회 (초안+검증)	12	greedy 토큰은 7 ms
행동 expert (Euler 5 × 후보 6)	70	

요청 지연 = 20 + 75 + 70 + 12 × r = **165 + 12 r** ms (r: 추측 라운드 수, 최소 1회).

r	롤링, 5단계	콜드 스타트 추정 (ViT 선형 가정)	롤링, 10단계 (PyTorch 기본값)
0	165	218	235
1	177	230	247
2	189	242	259
4	213	266	283
7	249	302	319

3.1 블로그 범위와 레시피 범위의 대응

- **블로그 160-190 ms**는 r = 0-2에 해당합니다. 하한 160 ms는 **라운드가 0개여도 단계 합 165 ms보다 낮습니다.** 레시피 단계 시간으로는 블로그의 하한을 설명할 수 없습니다.
- **레시피 180-250 ms** (10개 내부 검증 클립, 1-7 라운드)는 r = 2-7에 해당합니다. r = 1(177 ms)은 반올림해서 180으로 적은 것으로 보입니다.
- 레시피 README 본문 첫 줄은 "약 200 ms", 커밋 메시지는 "약 180-250 ms", 결과 절은 "예시 클립 약 220 ms"라고 씁니다. 예시 클립은 r = 4로 단계 합이 213 ms라 "약 220 ms"와 7 ms 차이입니다(반올림 또는 오버헤드 차이).

3.2 측정 조건이 지연을 얼마나 바꾸는가

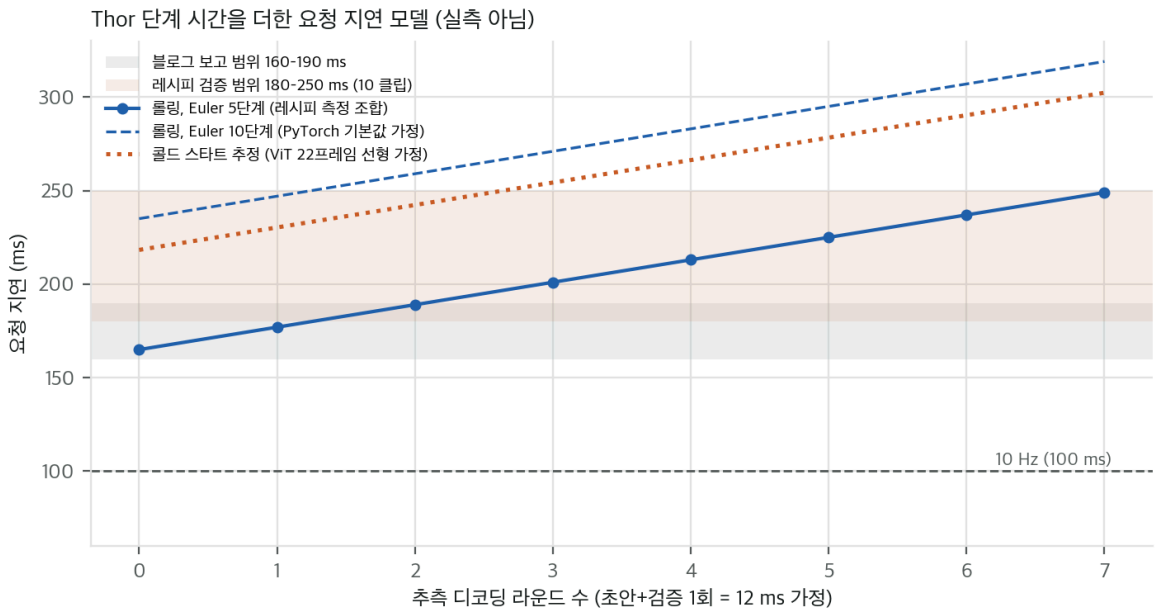


그림 1. 레시피 단계 시간으로 만든 요청 지연 모델(자체 계산). 롤링·콜드·Euler 단계 수에 따른 차이와 블로그·레시피 범위, 10 Hz 기준선.

- **롤링 대 콜드 스타트**: 레시피의 20 ms는 최신 6프레임만 인코딩한 값입니다. 22프레임을 모두 새로 인코딩하면 ViT가 약 3.7배(22/6, 선형 가정)가 되어 약 **53 ms가 늘어** 230 ms 근처가 됩니다. 실제 값은 측정이 필요합니다.
- **Euler 단계**: 공개 PyTorch 예제는 10단계, Thor 엔진은 5단계입니다. 단계를 10으로 늘리면 약 **70 ms**가 더 듭니다. 블로그 코드 예제는 10단계인데 Thor 수치는 5단계 기준이라, 블로그 안에서도 두 조건이 섞여 있습니다.
- **10 Hz (100 ms)**: 단계 합의 하한 177 ms보다 **77 ms 모자랍니다.** Embedl의 "10 Hz" 주장은 파이프라인 병렬화나 프레임 재사용 같은 추가 조건이 있어야 성립합니다. 블로그와 레시피에 그 조건은 없습니다.

3.3 NOMINAL 비율에 따른 평균 지연 (블로그 주장의 가정 민감도)

NOMINAL 프레임은 추론 토큰이 없다고 가정하면(r = 0), 평균 지연은 다음과 같습니다.

NOMINAL 비율 f	평균 지연 (ms)
0 (항상 CoT 2라운드)	189
0.5	177
0.9	167

NOMINAL 비율을 블로그는 보고하지 않았습니다. 이 민감도는 "적응형 추론이 평균 지연을 얼마나 줄일 수 있나"의 상한 감각만 줍니다. 최악(모든 프레임이 CoT)에서도 r = 2면 189 ms입니다.

4. 디코드 대역폭 상한 (메모리 바운드 가정)

토큰 하나를 만들 때 텍스트 디코더의 층 가중치와 LM head를 한 번씩 읽고, 컨텍스트 3,300 토큰의 KV를 읽는다고 가정했습니다.

형식	토큰당 읽는 바이트	상한 tok/s (273 GB/s)	상한 tok/s (200 GB/s)
BF16, 전체 head, BF16 KV	3.75 GB	73	53
FP8, 전체 head, BF16 KV	2.06 GB	132	97
FP8, 축소 head(8192행), FP8 KV (레시피)	1.61 GB	169	124

273 GB/s는 사전 지식이고 확인하지 않았습니다. Apple M4 Pro와 DRIVE/Jetson Thor 모두 이 값을 쓰지만, 공식 명세로 다시 확인해야 합니다. 200 GB/s 열은 보수적인 비교용입니다.

4.1 레시피의 7 ms/토큰이 말하는 것

레시피 greedy 토큰 7 ms를 레시피 형식(1.61 GB)으로 나누면 실효 대역폭은 **약 231 GB/s**입니다. 273 GB/s의 약 85%입니다. 디코드가 이미 메모리 대역폭 바운드에 가깝게 돌고 있다는 뜻입니다. 대역폭을 더 얻기 어렵다면 **바이트를 줄이는 것**(양자화, 축소 head, KV 양자화) 말고는 빠른 길이 없습니다.

4.2 대역폭 그림

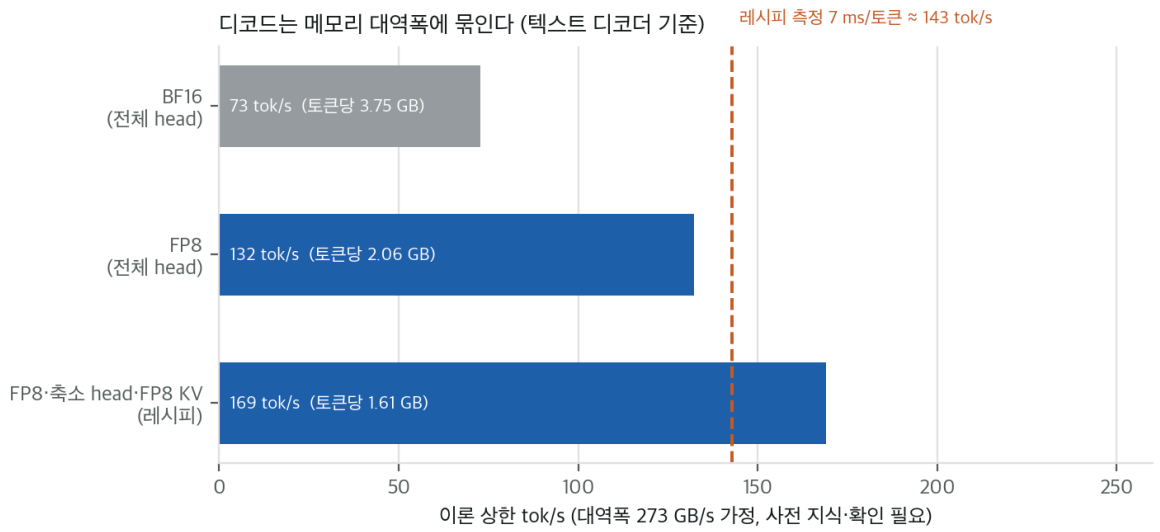


그림 2. 토큰당 읽는 바이트와 대역폭 상한(273 GB/s 가정). 레시피의 7 ms/토큰 위치를 함께 표시.

5. 추측 디코딩 수학 (블록 16, 초안 15개)

레시피의 방식은 다음과 같습니다. 초안 15개를 제안하고, 한 번의 검증 패스로 맞은 연속 길이 k와 교정 토큰 1개를 확정합니다. 라운드당 확정 토큰은 k + 1입니다. 수락률 α가 토큰마다 독립이라고 하면

$E(\alpha) = \Sigma_{k=0}^{15} \alpha^k$, greedy 대비 배율 = $E(\alpha) \times 7 / 12$

입니다. 몬테카를로(20,000회)와 식은 α ≤ 0.8에서 0.01 안에서 일치했고, α = 0.9에서는 몬테카를로 분산 때문에 0.09 차이가 났습니다.

α	E(α)	몬테카를로	greedy 대비 배율
0.2	1.25	1.25	0.73 (손해)
0.4	1.67	1.67	0.97
0.5	2.00	2.01	1.17
0.7	3.32	3.33	1.94
0.8	4.86	4.86	2.83
0.9	8.15	8.06	4.75

- **손익분기 수락률 α ≈ 0.42** ($E = 12/7 = 1.71$). 이보다 낮으면 라운드 비용(12 ms)이 greedy 토큰 여러 개(7 ms)보다 비싸서 손해입니다.
- 블로그는 "추측 디코딩으로 가속"이라고만 씁니다. 실제 수락률은 보고하지 않았습니다.

5.1 수락률 그림

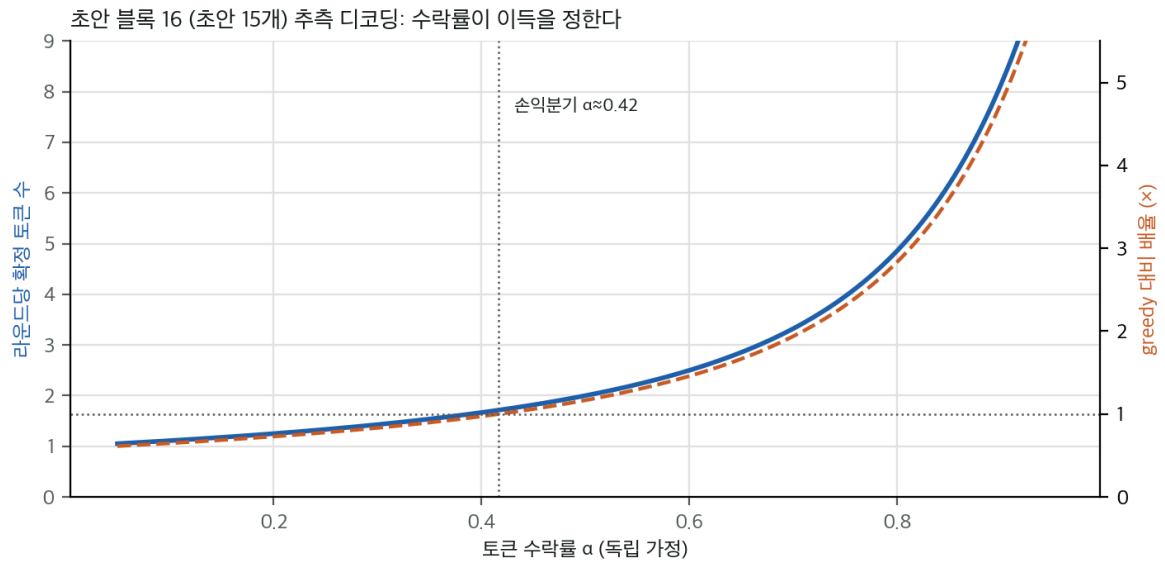


그림 3. 수락률 α에 따른 라운드당 확정 토큰 E(α)와 greedy 대비 배율(블록 16). 손익분기 α ≈ 0.42.

5.2 예시 클립의 4 라운드가 알려 주는 것

예시 클립의 CoT는 "Decelerate and turn right at the intersection to follow our route, as the roadway is clear."입니다. 토큰 수는 보고되지 않았지만 영어 한 문장이라 대략 20-30 토큰으로 추정합니다(확인 필요). 4 라운드에 이 길이를 만들려면 라운드당 약 6-7토큰이 필요하고, 시뮬레이션상 **α ≈ 0.9**에 가깝습니다. 토큰 수가 더 짧으면(약 17토큰) α ≈ 0.8로도 맞습니다. 즉 예시 클립의 수락률은 높았을 가능성이 큽니다. 다만 이것은 추정이며, 다른 클립에서는 달라질 수 있습니다.

5.3 CoT 길이에 따른 지연 (α = 0.8 가정, 롤링 5단계)

CoT 토큰 L	greedy (ms)	추측 α=0.8 라운드 수	추측 α=0.8 (ms)
10	235	2.8	198
25	340	5.9	236
50	515	11.1	296
100	865	21.3	420
256	1,957	53.5	806

CoT가 길수록 추측 디코딩의 이득이 커지고, 100 토큰이 넘으면 지연이 0.4초를 넘습니다. 로봇 VLA에서 행동 토큰을 자기회귀로 만들 때도 같은 비례 관계가 나타납니다.

6. AlpaJudge RF 점수 재계산

노트북에 적힌 부분점수(match 0.791, safe 0.987, grounded 0.994, complete 0.927, causal 0.852)로 공개 식을 다시 계산했습니다.

- action = p_match × (1 + p_safe) / 2 = **0.7859** (기록 0.787)
- component = mean(grounded, complete, causal) = **0.9243** (기록 0.924)
- overall = 0.6 × action + 0.4 × component = **0.8412** (기록 0.842)

노트북은 표시 점수를 "소수 셋째 자리에서 절사"한다고 적었고, 입력 부분점수는 소수 셋째 자리로만 적혀 있습니다. 절사로 보면 overall 가능 범위는 0.8412-0.8425이고, 반올림으로 보면 0.8406-0.8419입니다. **기록값 0.842는 두 규칙 모두와 양립합니다.** RB 0.999는 체크포인트 실행이 필요해서 확인하지 못했습니다.

7. 블로그 그림 수치의 산술 (Figure 2, Figure 5)

- 60배: 15.15 / 0.25 = 60.6. "약 60배"는 맞습니다. 다만 분모 0.25의 출처를 확인하지 못했습니다.
- LingoQA: Edge 75.2 대 Qwen 79.4. Edge가 **4.2점 낮습니다.** "모든 주요 과제에서 앞선다"는 제목과 맞지 않습니다.

- Figure 5: 기준선 보상의 자체 평가 13.4% 대 사람 평가 1.8%. 차이 11.6점. AlpaJudge 보상은 자체 평가가 사람보다 1.2점 낮습니다. 이 차이가 보상 설계의 효과인지 판정기 편향인지는 이 그림만으로 알 수 없습니다.
- 데이터 선별(Figure 4): 원자료에서 중간 구간 27%가 샘플링 뒤 81%가 됩니다. 샘플링 가중치 곡선의 정규화 방식은 블로그에 없습니다.

8. 기타 정합성 검사

검사	결과
카메라 프레임 수: 전방 2대 × 5 + 나머지 4대 × 3	22 (블로그·카드 22와 일치)
웨이포인트 64 × 0.1 s	6.4 s (카드와 일치)
과거 자세 20개, 0.1 s 간격, −1.9–0 s	20점 (일치)
궤적 토큰 57 = 3축 × 19차분	일치 (config `tokens_per_history_traj` =57)
레시피 어휘 2211개 → 8192행 패딩	패딩 후 행 수가 어휘보다 크다 (일치)
레시피 DFlash2 드래프터 블록 16, 제안 15	`speculative.py` 의 `block − 1` 구조와 일치

04. 주장 검증과 구현 가능성

이 문서는 블로그 주장 대조(§1), 공개 코드·설정의 정적 대조(§2), 확인 못 한 항목(§3), 구현 가능성 판정(§4)으로 이루어집니다. 가중치와 체크포인트는 받지 않았고, 공개 코드는 실행하지 않았습니다. `gh api`로 읽은 텍스트와 config.json만 사용했습니다.

판정 표기: **일치**, **부분 일치**(조건이 빠짐), **불일치**, **확인 불가**(공개 자료 없음 또는 실행 필요).

1. 주장 대조 (C1-C21)

#	블로그 주장	확인한 결과	판정	근거
C1	"2B-parameter" 모델	전체 2.649B (백본 2.45 + expert 0.2)	부분 일치 (공칭 명칭)	`03` §1
C2	Thor에서 픽셀→궤적 160-190 ms	레시피 단계 합 165+12r ms. 160은 r=0보다 낮음. 측정 조건 미공개	부분 일치, 조건 불명	`03` §3
C3	Figure 2: 페루프 중대 사고 간 거리 약 60배	15.15/0.25 = 60.6. 모델 카드 15.15 ± 0.04 km	일치 (분모 0.25 출처 불명)	모델 카드, `03` §7
C4	"Outperforms across major AV tasks" (Figure 2)	LingoQA: Edge 75.2 < Qwen 79.4	**불일치** (한 과제에서 낮음)	모델 카드 75.2
C5	Figure 2 Edge 값 (Ego3D, VLADBench, SURDS, WaymoQA, nuReasoning, KITScenes)	모델 카드 표와 0.1 이내 일치	일치	모델 카드 Table
C6	페루프 주행 성능 +48.9% (AlpaGym RL)	공개 수치 없음. 공개 정책은 Alpamayo 1.5 10B뿐	확인 불가	alpagym 트리
C7	Figure 5: AlpaJudge 보상 +10.2% (자체) / +11.4% (사람)	그림 판독값. 데이터 공개 없음	확인 불가 (그림 판독만)	`03` §7
C8	Figure 5: 기준선 보상 +13.4% (자체) / +1.8% (사람)	자체·사람 평가가 크게 어긋남	확인 불가 (그림 판독만), 해석에 주의	`03` §7
C9	AlpaJudge "human-calibrated"	보정 데이터, 일치도, 방법이 저장소에 없음	확인 불가	alpajudge README
C10	RF 점수 식: action = p_match(1+p_safe)/2, overall = 0.6·action + 0.4·component	`reference_free/model.py`와 일치	일치	rf_model.py
C11	노트북 RF 결과 overall 0.842	기록 부분점수로 재계산 0.8412. 절사·반올림 양쪽과 양립	일치	`03` §6
C12	노트북 RB alignment 0.999	체크포인트 실행 필요	확인 불가	—
C13	샘플링 설정: seed 43, top_p 0.98, temperature 0.6, 256 토큰, 10 Euler 단계	`inference.py` 기본값과 동일	일치	inference.py L278-285
C14	"" 특수 토큰으로 추론 생략	**학습 쪽은 확인:** alpamayo-recipes `scripts/prepare_pai_annotations.py`가 `<NOMINAL>` 라벨을 세고, "`<NOMINAL>` 라벨이 없으면 미세조정 중 행동이 바뀔 수 있다"는 경고를 둠. **추론 쪽은 미확인:** 공개 추론 코드에는 해당 문자열이 없고, 레시피 어휘는 ID 목록(2211)	부분 일치 (추론 빈도 확인 불가)	`02` §4
C15	6 카메라, 22 프레임, 카메라별 픽셀 예산	config와 README 일치	일치	config.json, README
C16	오프라인 증류: Alpamayo 2 Super 자동 라벨	증류 코드·데이터 없음. Super SFT 레시피는 있음	확인 불가	alpamayo-recipes
C17	FP8 양자화 + TensorRT·Edge-LLM 레시피	`quantize.py`, `action_graph.py`, 레시피 README와 일치 (FP8 KV, expert FP8 GEMM, ViT 앞 2층 FP16)	일치 (구현 존재)	`04` §2, D5
C18	DFlash2 추측 디코딩이 품질을 떨어뜨리지 않음	레시피: 10개 검증 클립에서 greedy 체인과 완전히 같음. 초안 모델은 릴리스에 없음	부분 일치 (자체 보고, 초안 모델 미공개)	레시피 README

#	블로그 주장	확인한 결과	판정	근거
C19	행동 expert 5 Euler 단계 (Thor)	레시피 README 확인. 블로그 코드 예제는 10단계	일치 (단, 블로그 안에서 조건이 섞임)	`03` §3
C20	Embedl: AGX Thor에서 10 Hz	10 Hz = 100 ms. 레시피 단계 합 하한 165-177 ms	**불일치** (추가 병렬화 없이는 불가)	`03` §3.2
C21	파트너 주장 (PORT 2배 계획 속도, Niulinx minADE 13.6%, 횡방향 46.1%)	파트너 자체 보고	확인 불가	블로그 인용

2. 공개 코드·설정 정적 대조 (D1-D10)

#	대조 항목	발견	함의
D1	치수: config.json ↔ config.py ↔ 블로그	텍스트 28층/2048, 비전 27층/1152, expert 28층/512로 일치	구조 주장은 신뢰 가능
D2	모델 리비전	PyTorch README는 `f8951d09…`, Thor 레시피 테스트는 `aaaa32c…`. 블로그 코드 예제는 리비전 없이 `load_model("nvidia/Alpamayo2-Edge")`	**두 리비전이 같은 가중치인지 확인 필요.** 재현 시 리비전을 명시해야 함
D3	지연 수치의 기재	블로그 160-190 ms, 레시피 README "약 200 ms", 결과 절 "180-250 ms (10 클립)", 커밋 메시지 "약 180-250 ms"	기재가 서로 다름. 블로그의 하한이 가장 낮음
D4	검증 클립 설명	레시피 README는 "10 internal validation clips", 커밋 메시지는 "ten public golden-bundle clips"	어느 쪽 클립인지 불명. 재현 자료 확인 필요
D5	추측 디코딩 구현	`thor/runtime/speculative.py` : 초안 block–1개, 검증 1회, 맞은 연속 길이 + 교정 1개	구현은 명확함. 블로그 수치와 조건 연결은 불명
D6	초안 모델 공개 여부	레시피 README: "DFlash2 block-16 drafter … not part of the model release"	**블로그 수치의 핵심 요소가 공개되지 않음.** 공개 저장소의 `speculative` 검색은 `alpamayo2-edge` 에서 0건
D7	PyTorch 가속 경로	`--accel tuned` : torch.compile, CUDA 그래프, FlashAttention 2(선택)	가속은 CUDA 전용
D8	샘플링 스텝	PyTorch 기본 10단계, Thor 5단계. 블로그 예제 10단계	같은 모델이라도 결과가 다를 수 있음 (궤적 차이를 재야 함)
D9	품질 비교 기준	레시피는 FP16 엔진과 minADE_6 비교(1.547 대 1.570 m, 10 클립). BF16 PyTorch와의 직접 비교는 없음	양자화 품질은 PyTorch 기준으로 보지 않았음
D10	RL 경로	alpagym 공개 정책은 `alpamayo_r1` (1.5, 10B). Edge용 RL 정책 없음. 블로그 Workflow 3의 그레이더 인터페이스(`BaseReasoningGrader`, `GRADER_REGISTRY`)는 `alpamayo1_x_rl` 에 있고 예시는 LingoJudge. AlpaJudge 어댑터는 없음	블로그의 RL 보상 흐름은 공개 코드로 **재구성만 가능**(어댑터 직접 작성)
D11	AlpaJudge API 이름	`load_reference_based`, `score_reference_based`, `load_reference_free`, `score_reference_free` 모두 존재	블로그 예제 코드는 정확함
D12	노트북 기록값	노트북 셀 출력은 지워졌고(`outputs: []`), 수치는 마크다운 셀에 적힘	실행으로 재현한 값이 아님
D13	모델 카드 vs 블로그	Edge 벤치마크 값, 2.45/0.2 B, 리비전, 16 GB 권장	일치
D14	라이선스	코드 Apache-2.0, 가중치 OpenMDW-1.1 (모델 카드, AlpaJudge README)	사용 조건은 가중치 쪽을 따라야 함
D15	개방 루프 minADE 값	모델 카드 minADE_6@5 s = 0.52 m (1,392 샘플). 레시피 minADE_6 = 1.55 m (10 클립, 기간 미표기)	두 수치는 데이터·기간이 달라 **비교할 수 없음**. 레시피 값을 정확도 주장으로 읽으면 안 됨

3. 확인하지 못한 항목 (실행 또는 추가 다운로드 필요)

1. AlpaJudge RB 0.999, RF 노트북 실행 결과 (체크포인트 실행)
2. NOMINAL 토큰의 추론 시 출현 빈도 (tokenizer.json 17.8 MB로 어휘 존재는 확인 가능, 빈도는 실행 필요)
3. 두 모델 리비전(`f8951d09` , `aaaa32c`)의 가중치 동일성 (HF 파일 해시 비교로 가능)
4. Thor 지연 160-190 ms의 측정 조건 (블로그 작성자 문의 필요)
5. 드래프터 DFlash2의 공개 여부와 수락률 (블로그·레시피 어디에도 수락률 수치 없음)
6. 양자화 품질 대 BF16 (PyTorch 기준 실행 필요)

7. RL +48.9%의 기준선과 시나리오 수
8. AlpaJudge 사람 보정 데이터와 일치도
9. 273 GB/s 대역폭 (M4 Pro, Thor) — 사전 지식, 공식 명세로 확인 필요
10. Qwen-Drive-1.0 (4B) 수치의 출처

4. 구현 가능성 판정

4.1 한 줄 판정

이 맥(Apple M4 Pro, 통합 메모리 24 GiB, Metal/MPS, CUDA 없음)에서는 블로그의 온보드 시스템(Thor 실시간 160-190 ms)을 구현할 수 없습니다. 모델의 기능적 PyTorch 추론은 메모리상 들어가고, 느린 형태로 **포팅은 가능(조건부)**합니다. Thor 엔진, FP8 보정, 추측 디코딩 초안 모델은 하드웨어와 공개되지 않은 부품 때문에 불가능합니다.

4.2 항목별 판정

항목	필요 자원	이 맥 판정	근거·추정	노력 (추정)
공식 PyTorch 추론 (Edge, 22 프레임 → CoT + 궤적)	CUDA GPU 16 GB 이상 권장, BF16, transformers 5.17, SDPA 또는 FA2	**조건부 가능**: 가중치 BF16 4.9 GiB는 들어감. `device="cuda"`, `torch.Generator(device="cuda")`, `autocast("cuda")`를 MPS 분기로 바꾸고 BF16 수치 차이를 확인해야 함	프리필 약 12.7 TFLOP (LM 1.96B × 2 × 3236). 맥 GPU 실효 수 TFLOPS 가정 → **수 초 (추정 2-4 s)**. 지연 목표 0.2 s 대비 10배 이상	포팅 1-2주 (가정), 수치 검증에 GPU 필요
블로그 지연 목표 재현 (160-190 ms)	Thor 또는 H100급	**불가**	프리필만 맥에서 수 초	—
디코드 (CoT 25 토큰)	메모리 대역폭	**가능하지만 느림**	BF16 상한 약 73 tok/s (273 GB/s 가정) → 25토큰 약 0.35 s	—
AlpaJudge RB (텍스트 전용)	CUDA 명시, 가중치 약 4-5 GB	**조건부 가능**	텍스트 모델 2B급. CPU 또는 MPS 실행은 미확인	포팅 수 일 (가정)
AlpaJudge RF (6프레임 + 텍스트)	GPU 권장, 가중치 약 5 GB	**조건부 가능**	비디오 프레임 디코딩과 PAI 데이터 접근(게이트) 필요	포팅 1주 (가정)
Thor TensorRT 엔진 빌드·실행	DRIVE AGX Thor, TensorRT 11.3 SDK, DRIVE OS	**불가**	보드와 자동차용 SDK 필요	—
FP8 보정 ·ONNX 내보내기	x86 호스트 + NVIDIA GPU 16 GB 이상, ModelOpt 0.45, TensorRT Edge-LLM	**불가**	CUDA·ModelOpt 전용	—
추측 디코딩 초안 모델 학습	공개되지 않음. 유사 레시피는 80 GB GPU 8장	**불가** (현재 조건)	`alpamayo1_5_speculative_decoding`은 8× 80 GB 기준	—
폐루프 RL (AlpaGym)	멀티 GPU, AlpaSim	**불가**	공개 정책이 Edge가 아님	—
데이터 선별 (판정기 점수 기반)	PAI 데이터 접근, 판정기 추론	**조건부**	판정기 추론이 가능해야 함	판정기 배치 실행 시간에 비례
산술·toy 재현 (본 폴더 스크립트)	Python, numpy	**가능, 완료**	이 저장소의 `analysis/*.py`	완료

4.3 "구현할 수 있는가"에 대한 답

- **재구현(개념·산술·수식)**: 가능. 지연 모델, 추측 디코딩 수학, RF 점수식, 대역폭 상한은 이미 맥에서 재계산했습니다.

- **기능 포팅(맥에서 같은 모델이 같은 출력을 내는지)**: 조건부. 메모리는 충분하고 CUDA 호출만 고치면 되지만, 수치 검증에는 GPU 기준값이 필요합니다. 승인과 다운로드가 필요합니다.
- **성능 구현(160-190 ms)**: 불가. 이 맥에서는 같은 모델이라도 수 초가 걸릴 것으로 추정됩니다. Thor 엔진과 드래프터가 없으면 블로그 수치에 가까워질 수 없습니다.
- **라이선스·접근**: 가중치는 OpenMDW-1.1이고 PAI 데이터는 게이트가 있습니다. 실행 전에 사용 조건을 확인해야 합니다.

4.4 맥에서 할 수 있는 다음 단계 (승인 필요)

1. 가중치 없이 할 수 있는 것: tokenizer.json 검사(NOMINAL 확인, 17.8 MB)와 두 리비전의 해시 비교(D2).
2. 설치 승인 후: torch와 transformers 5.17을 맥에 설치하고, 모델 로드 대신 **config 기반 메모리 측정**(빈 모델 생성, meta device)으로 포팅 가능성 확인.
3. 가중치 다운로드 승인 후: RB 체크포인트(약 4 GB)를 맥 MPS에서 실행해 노트북의 RB 0.999와 비교.

5. 검증 방식 요약

- 수치 재계산: config, 레시피 단계 시간, 노트북 기록값으로 계산 (`analysis/*.py`)
- 공개 코드 대조: 블로그 코드 예제와 `inference.py` , 점수 식과 `rf_model.py` , 레시피 `speculative.py` · `quantize.py`
- 실행: 하지 않음. 가중치 다운로드 없음